

회귀분석 예제

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
import statsmodels.api as sm

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image
```

단순회귀분석

데이터 불러오기

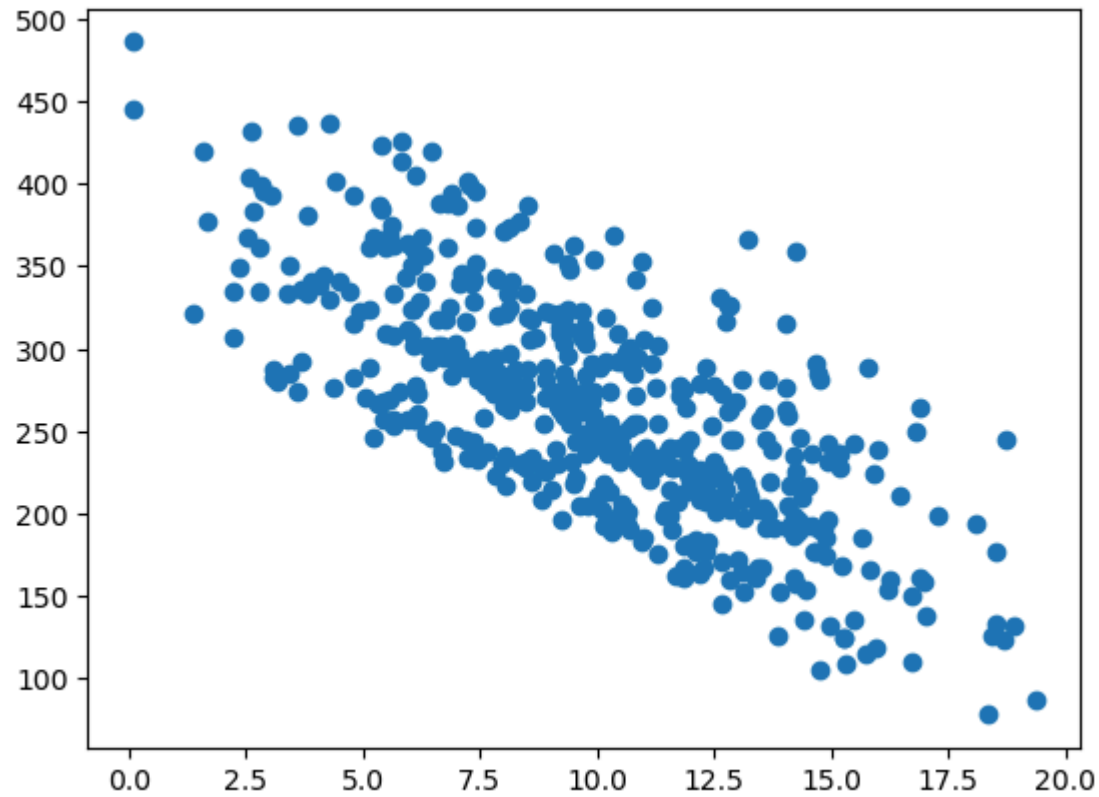
```
In [2]: used_car = pd.read_csv("used_car.csv")
used_car.head()
```

```
Out[2]:
```

	odometer	year	size	price
0	12.18	6	1	163.61
1	11.71	4	1	206.77
2	5.81	2	4	414.24
3	5.24	3	3	367.94
4	13.66	7	4	281.38

산점도를 그려 선형관계 파악하기

```
In [3]: plt.scatter(data = used_car, x = 'odometer', y = 'price')
plt.show()
```



회귀모형 만들기

```
In [4]: '''
`summary()` 함수의 리턴값은 선형 회귀 모델의
결과를 요약한 테이블입니다.
이 테이블에는 모델의 통계적인 특징과 모델의 성능에 관한
여러 가지 정보들이 포함됩니다.
1. Dep. Variable: 종속 변수의 이름을 나타냅니다.
   여기서는 'price'가 종속 변수입니다.
2. Model: 모델의 종류를 나타냅니다.
   일반적으로 OLS(Ordinary Least Squares) 회귀 모델을
```

사용했을 때는 'OLS'로 표시됩니다.

3. **Method:** 모델 적합에 사용된 추정 방법을 보여줍니다.
4. **No. Observations:** 모델에 사용된 관측치의 수를 나타냅니다.
5. **DF Residuals:** 잔차의 자유도를 나타냅니다.
6. **Df Model:** 모델의 독립 변수의 수를 나타냅니다.
7. **Covariance Type:** 공분산의 타입을 나타냅니다.
8. **R-squared:** 결정 계수(R-squared)는 모델이 종속 변수의 분산을 설명하는 비율을 나타냅니다.
종속 변수의 분산을 모델로 설명하는 비율이 높을수록 모델이 데이터에 더 잘 적합되었다고 할 수 있습니다.
9. **Adj. R-squared:** 수정된 결정 계수(Adjusted R-squared)는 독립 변수의 수와 표본 크기를 고려하여 결정 계수를 보정한 값입니다.
보정된 결정 계수는 독립 변수의 수가 증가하면서 발생하는 모델의 과적합을 보정하는 데 사용됩니다.
10. **F-statistic:** F 통계량은 회귀 모델의 적합도를 평가하는 데 사용됩니다.
F 통계량이 크고 p-value가 작으면 모델이 통계적으로 유의미하다고 할 수 있습니다.
11. **Prob (F-statistic):** F 통계량의 p-value 값입니다.
모델이 통계적으로 유의미한지 여부를 판단하는 데 사용됩니다.
12. **Log-Likelihood:** 로그 우도(Log-Likelihood)는 모델이 주어진 데이터에서 발생할 가능성의 로그입니다.
로그 우도가 높을수록 모델이 데이터에 더 적합해진다고 할 수 있습니다.
13. **AIC:** AIC(Akaike Information Criterion)는 모델의 상대적인 품질을 나타내는 지표입니다.
낮은 AIC 값은 더 나은 모델을 의미합니다.
14. **BIC:** BIC(Bayesian Information Criterion)는 AIC와 유사하지만, 모델의 복잡성에 대해 더 엄격한 페널티를 적용합니다.
15. **Omnibus:** Omnibus는 잔차의 정규성을 검정하는 통계량입니다.
16. **Prob(Omnibus):** Omnibus 검정의 p-value 값입니다.
잔차가 정규 분포를 따르는지 여부를 판단하는 데 사용됩니다.
17. **Durbin-Watson:** Durbin-Watson 통계량은 잔차 간의 자기 상관을 평가하는 지표입니다.
값이 2에 가까우면 자기 상관이 없다고 가정할 수 있습니다.
18. **Jarque-Bera (JB):** Jarque-Bera 통계량은 잔차의 정규성을 검정하는 다른 방법입니다.
19. **Prob(JB):** Jarque-Bera 검정의 p-value 값입니다.
잔차가 정규 분포를 따르는지 여부를 판단하는 데 사용됩니다.
20. **Cond. No.:** 조건 수(Condition Number)는 회귀 분석에서 다중공선성을 평가하는 지표입니다.
조건 수가 크면 다중공선성의 가능성이 높다고 판단됩니다.

이러한 정보들은 주어진 데이터에 대한 선형 회귀 모델의 적합도를 평가하고, 모델이 데이터를 얼마나 잘 설명하는지를 파악하는 데 도움을 줍니다.

```
'''
```

'''

`used_car.lm`은 선형 회귀 모델을 나타내는 변수입니다.
이 코드에서는 `StatsModels` 패키지의 `ols()` 함수를 사용하여 주어진 데이터에 대한 선형 회귀 모델을 적합시키고, 그 결과를 `used_car.lm`에 저장하고 있습니다.

여기서 `ols()` 함수는 "Ordinary Least Squares"의 약자로, 최소제곱법을 이용하여 선형 회귀 모델을 적합시키는 함수입니다.
첫 번째 인자로 종속 변수와 독립 변수들의 관계를 나타내는 `formula`를 전달하고, 두 번째 인자로 데이터가 포함된 `DataFrame` 객체를 전달합니다.

`fit()` 메서드는 모델을 데이터에 적합시키는 역할을 합니다.
이를 통해 선형 회귀 모델이 주어진 데이터에 대해 계산되고, 모델의 계수(`coefficients`) 및 다른 통계적 특성들이 추정됩니다.

그런데 `used_car`은 `DataFrame` 객체이며, 여기에 `.lm`이라는 필드를 생성하는 것은 좋은 프로그래밍 습관이 아닙니다.
올바른 방법은 새로운 변수를 만들어서 모델을 저장하는 것입니다.

```
'''
```

```
used_car.lm = smf.ols('price ~ odometer', data=used_car).fit()  
used_car.lm.summary()
```

Out[4]:

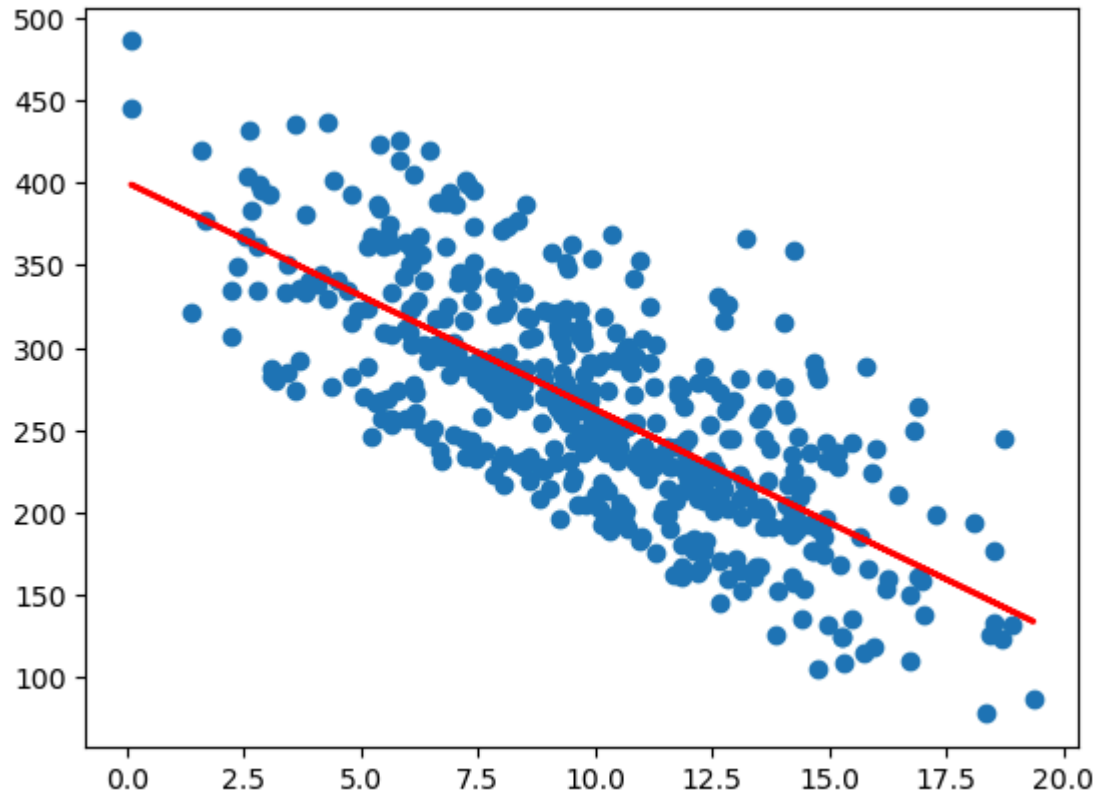
OLS Regression Results						
Dep. Variable:		price		R-squared:		0.531
Model:		OLS		Adj. R-squared:		0.530
Method:		Least Squares		F-statistic:		564.2
Date:		Fri, 01 Mar 2024		Prob (F-statistic):		5.90e-84
Time:		23:19:48		Log-Likelihood:		-2635.1
No. Observations:		500		AIC:		5274.
Df Residuals:		498		BIC:		5283.
Df Model:		1				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
Intercept	400.3163	6.132	65.288	0.000	388.269	412.363
odometer	-13.7601	0.579	-23.754	0.000	-14.898	-12.622
Omnibus:		16.755	Durbin-Watson:		2.102	
Prob(Omnibus):		0.000	Jarque-Bera (JB):		17.235	
Skew:		0.430	Prob(JB):		0.000181	
Kurtosis:		2.702	Cond. No.		31.0	

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

산점도에 회귀모형 그리기

```
In [5]: plt.scatter(data = used_car, x = "odometer", y = "price")
# col = 'r': 직선색을 붉은 색으로, linewidth = 2: 직선 두께를 2로
plt.plot(used_car.odometer, used_car.lm.predict(), color = 'r', linewidth = 2)
plt.show()
```



새로운 데이터 예측하기

```
In [6]: x_new = pd.DataFrame([10, 30], columns=['odometer'])
```

```
In [7]: used_car.lm.predict(x_new)
```

```
Out[7]: 0    262.715365
        1   -12.486506
        dtype: float64
```

```
In [2]: Image('2.png')
```

Out[2]: Multiple Regression : 다중회귀분석

1. 종속변수 y : 만성폐쇄성폐질환 (COPD) 삶의 질 평가 검사(CAT) 증상점수

- COPD 평가검사(COPD assessment test, CAT)
- CAT는 8가지 설문 사항을 통해 환자의 삶의 질 즉, 호흡기 증상, 활동 정도, 수면, 자신감을 지표로 삼고 평가
- 0(삶의 질 가장 좋음)~40점(삶의 질 가장 나쁨)까지의 점수로 나타남

2. 독립변수 x (7개)

- age : 나이, 연속형
- sex : 성별(1:남성, 0:여성), 범주형
- FEV1%: 1초간 노력성 폐활량(Forced Expiratory Volume in One second, FEV1), 연속형,
 - (> 70 :mild, 60-69 :moderate, 50-59:moderagely severe, 35-49:severe, <35 :very severe)
- Smoke_pack_year : 갑년(pack-year), 사람의 담배 노출을 측정하는 데 사용되는 지표, 하루평균 담배소비량(갑)×흡연기간(년)을 의미
- Chol : 혈중 콜레스테롤 (mg / dl 단위), 연속형
- Comorbid : 동반질환여부 (1:유, 0:무), 범주형
- premium : 월평균 건강보험료(소득수준 의미), 연속형
- COPD 환자의 80% 이상에서 10갑년 이상의 흡연력. 을 가지며, 특히 남자에서는 90% 이상 흡연과 관련

```
In [8]: df = pd.read_csv('copdcat.csv')
df
```

Out[8]:

	age	sex	FEV1%	Smoke_pack_year	Chol	Comorbid	premium	CATScore
0	53	1	74.13	9.2	170.83	1	59649	7.8
1	48	1	53.24	0.0	168.82	0	47348	9.1
2	55	1	70.39	16.3	193.98	0	49599	7.6
3	62	0	69.87	44.8	215.33	1	58428	14.5
4	48	1	76.41	32.8	184.65	1	50806	11.1
...	...	...	...	...	...	...	...	...
5246	45	1	53.32	0.0	241.90	1	45933	7.1
5247	37	0	57.03	44.4	182.58	0	38004	16.8
5248	56	0	50.61	18.6	198.92	0	66896	7.3
5249	53	1	63.59	45.3	206.71	1	67557	12.2
5250	42	1	64.52	11.9	200.45	0	51339	5.5

5251 rows × 8 columns

```
In [9]: X=df.drop(["CATScore"],axis=1)
        y=df["CATScore"]
```

In [10]:

```
'''
코드는 다음과 같은 작업을 수행합니다:
```

1. `df.drop(["CATScore"], axis=1)`: DataFrame인 `df`에서 "CATScore" 열을 삭제한 새로운 DataFrame을 생성합니다. 이를 `X` 변수에 할당합니다. 따라서 `X`는 "CATScore"를 제외한 모든 독립 변수들을 포함하고 있습니다.
2. `df["CATScore"]`: DataFrame인 `df`에서 "CATScore" 열을 선택하여 종속 변수로 사용합니다. 이를 `y` 변수에 할당합니다.
3. `sm.OLS(y, sm.add_constant(X))`: StatsModels 라이브러리에서 Ordinary Least Squares(OLS) 방법을 사용하여 선형 회귀 모델을 만듭니다. `y`는 종속 변수이고, `X`는 독립 변수입니다.

`sm.add_constant(X)`는 독립 변수에 상수항을 추가한 것입니다.
상수항을 추가하는 이유는 회귀 모델의 절편을 포함하기 위함입니다.

4. `model_reg.fit()`: `model_reg`에 대해 OLS 회귀 모델을 적합(`fit`)시킵니다.
이는 주어진 독립 변수와 종속 변수 사이의 관계를 나타내는
최적의 회귀식을 찾는 과정입니다.
5. `result_reg.summary()`: 적합된 회귀 모델에 대한 요약 통계 정보를 출력합니다.
이를 통해 모델의 성능 및 통계적 유의성을 평가할 수 있습니다.

...

```
model_reg = sm.OLS(y,sm.add_constant(X))
result_reg = model_reg.fit()
result_reg.summary()
```

Out[10]:

OLS Regression Results							
Dep. Variable:	CATScore		R-squared:	0.740			
Model:	OLS		Adj. R-squared:	0.740			
Method:	Least Squares		F-statistic:	2130.			
Date:	Fri, 01 Mar 2024		Prob (F-statistic):	0.00			
Time:	23:19:48		Log-Likelihood:	-10503.			
No. Observations:	5251		AIC:	2.102e+04			
Df Residuals:	5243		BIC:	2.107e+04			
Df Model:	7						
Covariance Type:	nonrobust						
	coef	std err	t	P> t	[0.025	0.975]	
const	9.7637	0.246	39.756	0.000	9.282	10.245	
age	0.0147	0.003	4.733	0.000	0.009	0.021	
sex	0.2690	0.049	5.446	0.000	0.172	0.366	
FEV1%	-0.0787	0.002	-50.431	0.000	-0.082	-0.076	
Smoke_pack_year	0.1542	0.001	107.823	0.000	0.151	0.157	
Chol	0.0003	0.001	0.387	0.699	-0.001	0.002	
Comorbid	0.9392	0.049	19.006	0.000	0.842	1.036	
premium	-1.504e-05	1.66e-06	-9.058	0.000	-1.83e-05	-1.18e-05	
Omnibus:	1902.692	Durbin-Watson:	1.987				
Prob(Omnibus):	0.000	Jarque-Bera (JB):	20452.640				
Skew:	1.418	Prob(JB):	0.00				

Kurtosis:	12.243	Cond. No.	5.68e+05
------------------	--------	------------------	----------

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 5.68e+05. This might indicate that there are strong multicollinearity or other numerical problems.

이상점, 영향점

```
In [11]: '''
`get_influence()` 함수는 StatsModels 패키지에서 제공되는
선형 회귀 모델의 영향력을 분석하는 메서드입니다.
이 메서드는 모델에 대한 영향력을 확인하고
이상치나 극단값(outliers)을 식별하는 데 사용됩니다.

사용법:
influence = model.get_influence()

인자:
- `model`: `fit()` 메서드로 적합된 StatsModels의 회귀 모델 객체입니다.

리턴값:
- `influence`: 선형 회귀 모델에 대한 영향력을 분석하는 객체입니다.
  이 객체에는 여러 가지 메서드와 속성이 있습니다.

특징:
- `get_influence()` 메서드는 선형 회귀 모델에 대한
  영향력을 분석하기 위한 객체를 반환합니다.
- 반환된 `influence` 객체를 통해 이상치나 극단값을 식별하고,
  모델의 적합도를 평가할 수 있습니다.
- `influence` 객체에는 다양한 메서드와 속성이 있으며,
  이를 사용하여 모델에 대한 다양한 정보를 얻을 수 있습니다.
  일반적으로는 이상치나 영향력 있는 관측치를 식별하고,
  이를 통해 모델의 적합성을 개선할 수 있습니다.
```

이상치나 극단값은 회귀 분석 결과에 부정적인 영향을 미칠 수 있으므로, 이를 식별하고 처리하는 것이 중요합니다.

``get_influence()`` 메서드를 사용하여 이를 확인하고 적절한 조치를 취할 수 있습니다.

```
'''
```

'''

``summary_frame()`` 함수는 `StatsModels` 패키지에서 제공되는 메서드로, 회귀 분석 결과를 `DataFrame` 형식으로 반환하는 기능을 제공합니다.

사용법:

```
summary_df = result.summary_frame()
```

인자:

- ``result``: ``fit()`` 메서드로 적합된 회귀 모델의 결과 객체입니다.

리턴값:

- ``summary_df``: 회귀 분석 결과를 담은 `DataFrame` 객체입니다.

특징:

- ``summary_frame()`` 메서드는 회귀 분석 결과를 `DataFrame` 형식으로 반환합니다.
이를 통해 결과를 더 쉽게 탐색하고 다양한 분석에 활용할 수 있습니다.
- 반환된 `DataFrame`에는 회귀 모델의 결과에 관한 여러 정보들이 포함되어 있습니다.
이는 표로 정리되어 있어 쉽게 읽고 분석할 수 있습니다.
- `DataFrame`에는 회귀 모델의 계수(`coefficients`), 표준 오차(`standard errors`), t-통계량(`t-statistics`), `p-value` 등의 정보가 포함됩니다.
- 추가적으로, 계수의 신뢰 구간(`confidence intervals`)과 결정 계수(`R-squared`), 수정된 결정 계수(`Adjusted R-squared`) 등의 통계적 지표도 포함될 수 있습니다.
- 이 메서드를 통해 회귀 모델의 결과를 데이터프레임 형태로 변환하면, 데이터프레임의 다양한 기능을 활용하여 회귀 분석 결과를 시각화하거나 추가적인 분석을 수행할 수 있습니다.

``summary_frame()`` 메서드를 사용하면 회귀 분석 결과를 쉽게 다룰 수 있으며, 데이터프레임을 이용한 다양한 분석 및 시각화 작업을 효과적으로 수행할 수 있습니다.

```
'''
```

```
result_influ = result_reg.get_influence().summary_frame() #slow
result_influ
```

Out[11]:

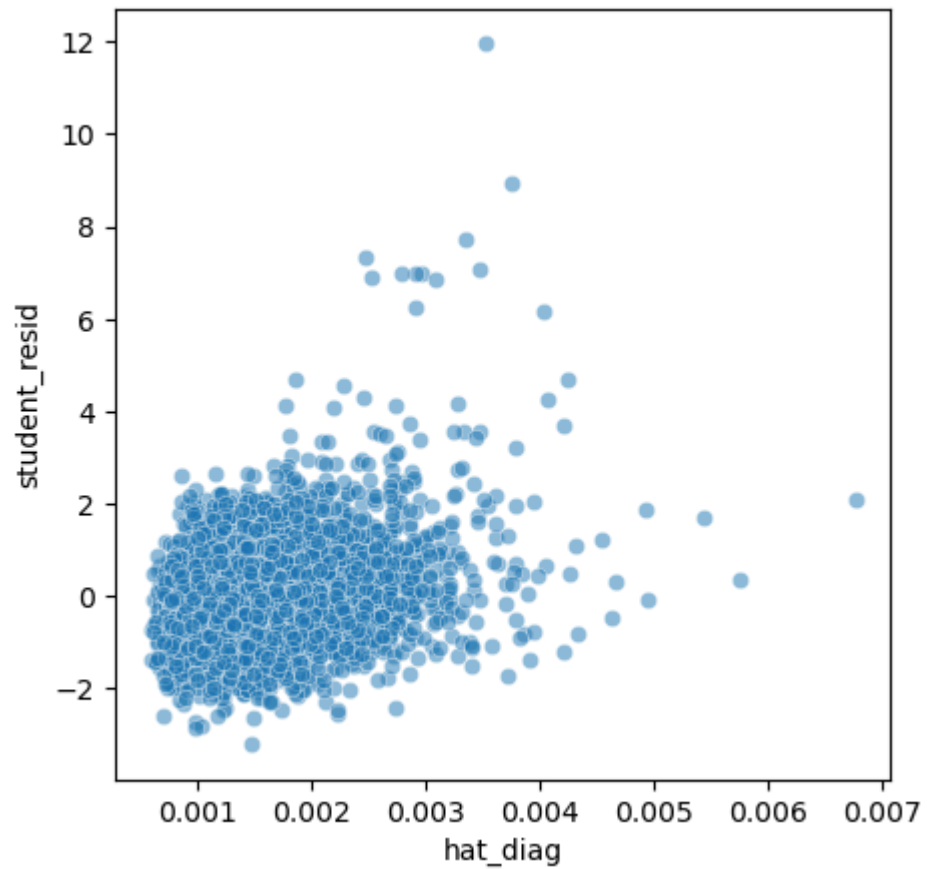
	dfb_const	dfb_age	dfb_sex	dfb_FEV1%	dfb_Smoke_pack_year	dfb_Chol	dfb_Comorbid	dfb_premium	cooks_d	standard_res
0	-0.003124	0.004385	0.010310	0.013619	-0.006759	-0.009415	0.009749	0.002819	0.000083	0.7364
1	0.028091	-0.004868	0.025208	0.001156	-0.031895	-0.024043	-0.026036	-0.013566	0.000460	1.8003
2	-0.001345	0.002557	0.003772	0.004261	-0.001104	-0.000988	-0.003864	-0.001557	0.000009	0.2684
3	-0.030141	0.027986	-0.017474	0.019802	0.025028	0.005634	0.017702	0.003660	0.000342	1.2848
4	-0.001531	-0.001492	0.008555	0.012861	0.005823	-0.004389	0.008178	-0.002385	0.000055	0.6112
...	...	...	...	...	...	...	...	...	...	...
5246	0.000089	-0.001296	0.002283	0.000058	-0.002778	0.002387	0.002124	-0.001415	0.000004	0.1624
5247	0.062065	-0.055527	-0.035466	0.012481	0.045790	-0.020056	-0.036533	-0.039964	0.001428	2.6040
5248	0.003137	-0.007471	0.009201	0.001202	0.001528	0.001388	0.009462	-0.007158	0.000047	-0.6809
5249	0.005108	-0.001903	-0.004494	-0.003108	-0.006317	-0.000382	-0.004424	-0.003801	0.000016	-0.3193
5250	-0.006113	0.008709	-0.009368	-0.006998	0.005225	0.000746	0.009574	0.002516	0.000052	-0.6640

5251 rows × 14 columns



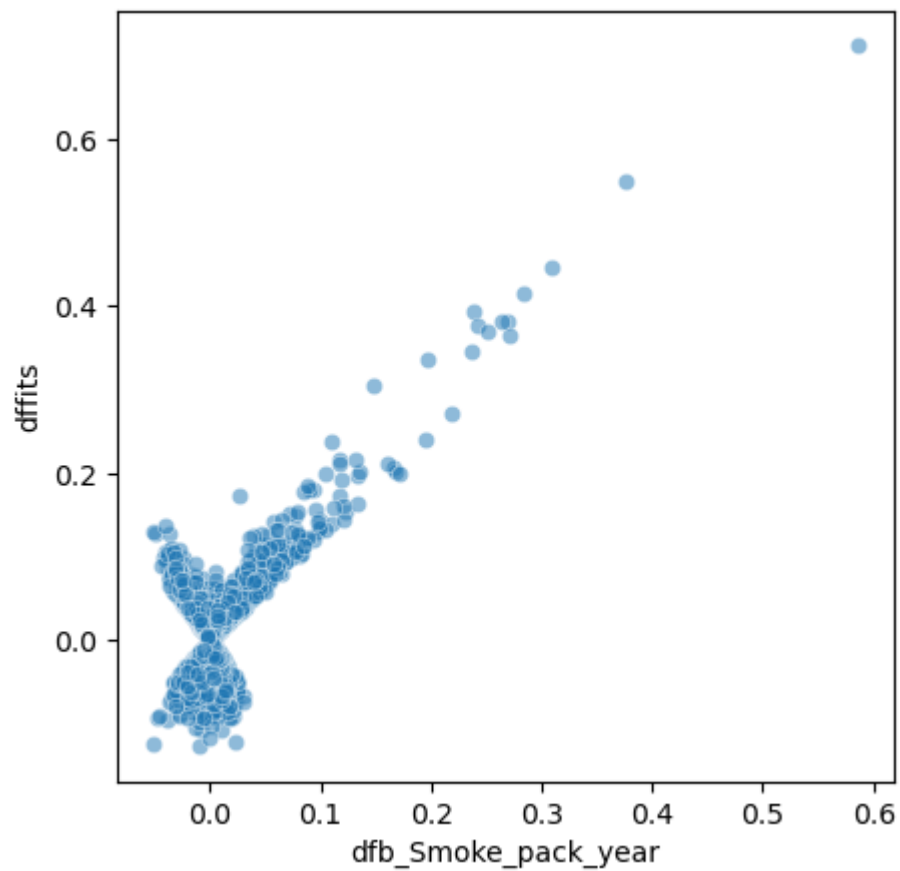
Leverage Hii, Studentized Residual

```
In [12]: plt.figure(figsize=(5,5))
sns.scatterplot(data=result_influ, y="student_resid", x="hat_diag",
                alpha=0.5)
plt.show()
```



Dffits, DFBETAS (FEV1%)

```
In [13]: plt.figure(figsize=(5,5))
sns.scatterplot(data=result_influ, y="dffits", x="dfb_Smoke_pack_year",
               alpha=0.5)
plt.show()
```



영향점 상위 10개

```
In [14]: result_influ_sort = result_influ.sort_values('student_resid',  
                                                    ascending=False)  
result_influ_sort[['student_resid', 'hat_diag', 'dffits', 'dfb_Smoke_pack_year']].head(10)
```

Out[14]:

	student_resid	hat_diag	dffits	dfb_Smoke_pack_year
3110	11.968086	0.003532	0.712515	0.586569
3405	8.937631	0.003759	0.548991	0.375621
4311	7.705960	0.003348	0.446633	0.309232
4032	7.324662	0.002477	0.364998	0.270733
2184	7.062071	0.003467	0.416570	0.284163
4947	6.998953	0.002789	0.370139	0.251364
1063	6.987678	0.002961	0.380804	0.268723
3489	6.980149	0.002907	0.376909	0.241177
2252	6.896445	0.002519	0.346549	0.236534
4197	6.856656	0.003088	0.381630	0.264482

다중공선성

```
In [15]: selling = pd.read_csv('selling.csv')
selling
```

Out[15]:

	Price	Bedrooms	HSize	LotSize
0	124100	3	1290	3900
1	218300	4	2080	6600
2	117800	3	1250	3750
3	168300	3	1550	4650
4	120400	3	1360	4050
...	...	...	...	...
95	180500	3	1320	3900
96	218100	5	2980	9000
97	117500	3	1570	4950
98	157400	3	1560	5100
99	155900	4	1620	4800

100 rows × 4 columns

```
In [16]: selling.lm = smf.ols('Price ~ Bedrooms+HSize+LotSize', data=selling).fit()  
selling.lm.summary()
```

Out[16]:

OLS Regression Results

Dep. Variable:		Price		R-squared:		0.560
Model:		OLS		Adj. R-squared:		0.546
Method:		Least Squares		F-statistic:		40.73
Date:		Fri, 01 Mar 2024		Prob (F-statistic):		4.57e-17
Time:		23:19:56		Log-Likelihood:		-1152.6
No. Observations:		100		AIC:		2313.
Df Residuals:		96		BIC:		2324.
Df Model:		3				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
Intercept	3.772e+04	1.42e+04	2.661	0.009	9576.984	6.59e+04
Bedrooms	2306.0808	6994.192	0.330	0.742	-1.16e+04	1.62e+04
HSize	74.2968	52.979	1.402	0.164	-30.865	179.458
LotSize	-4.3638	17.024	-0.256	0.798	-38.156	29.429
Omnibus:		0.204	Durbin-Watson:		1.926	
Prob(Omnibus):		0.903	Jarque-Bera (JB):		0.296	
Skew:		0.102	Prob(JB):		0.862	
Kurtosis:		2.829	Cond. No.		3.55e+04	

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.55e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
In [17]: '''
`pairplot()` 함수는 seaborn 라이브러리에서 제공하는
산점도 행렬(Scatterplot Matrix)을 그리는 함수입니다.
이 함수를 사용하면 다변량 데이터의 각 변수 간의 관계를 시각화할 수 있습니다.

사용법:
import seaborn as sns
sns.pairplot(data, hue=None, palette=None, ...)

인자:
- `data` (DataFrame): 시각화할 데이터프레임입니다.
- `hue` (옵션): 범주형 변수를 지정하여 그룹별로 색상을 구분할 수 있습니다.
- `palette` (옵션): 색상 팔레트를 지정합니다.
- 기타 다양한 옵션들이 있습니다. 예를 들어, `vars` 옵션을 사용하여
  특정 변수들만 선택하거나, `kind` 옵션을 사용하여
  다른 종류의 그래프를 그릴 수도 있습니다.

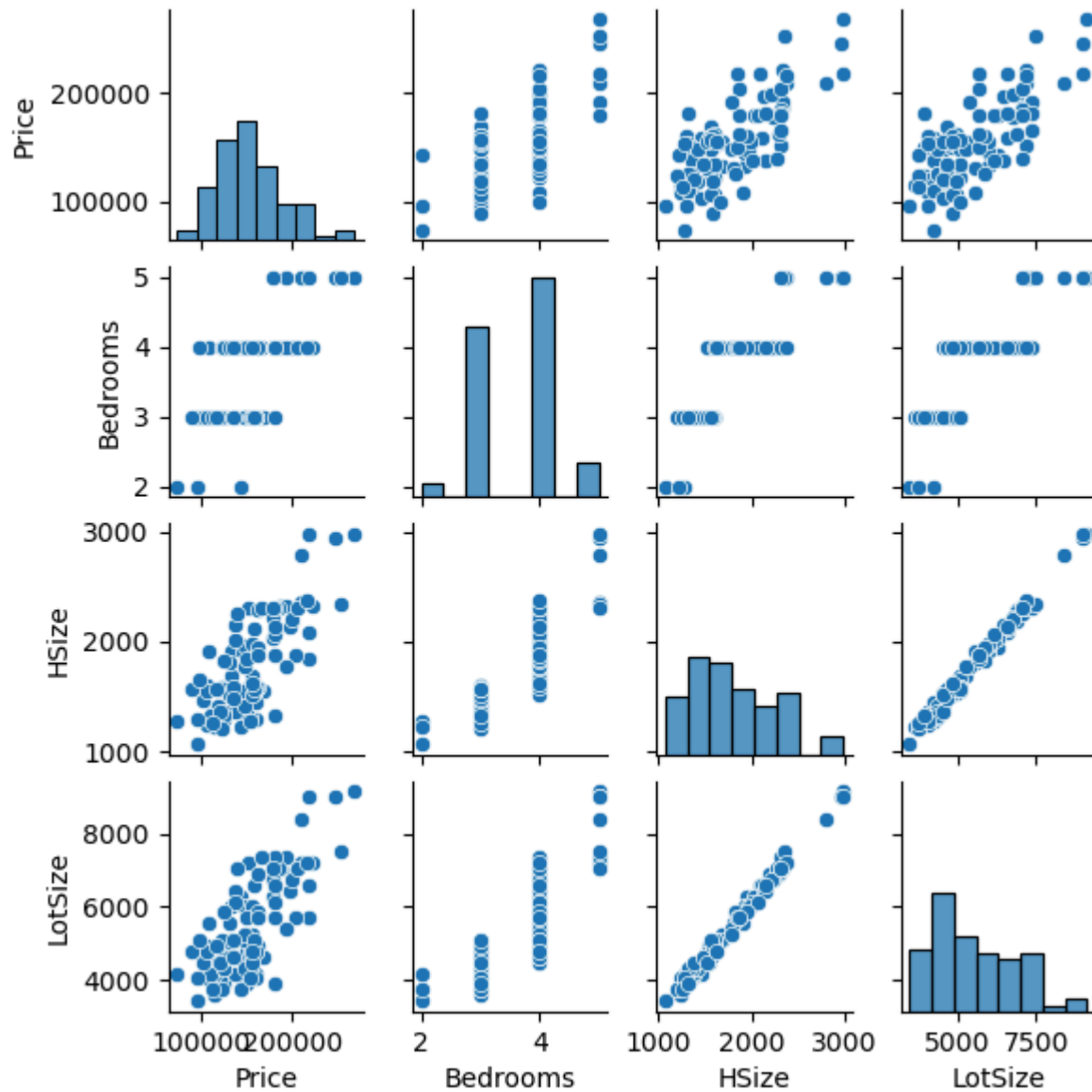
리턴값:
- 그림 객체를 반환합니다. 이를 사용하여 추가적인 조작이 가능합니다.

특징:
- `pairplot()` 함수는 주어진 데이터프레임의
  모든 변수들을 짝지어 만든 산점도를 그립니다.
- 대각선에는 각 변수의 분포를 히스토그램으로 표시합니다.
- 산점도 행렬은 변수들 간의 관계를 한눈에 파악할 수 있도록 도와줍니다.
- `hue` 옵션을 사용하면 범주형 변수를 기준으로 데이터를 그룹화하고,
  각 그룹을 다른 색상으로 표시할 수 있습니다.
- `palette` 옵션을 사용하여 색상 팔레트를 지정하여
  그룹을 시각적으로 구분할 수 있습니다.

`pairplot()` 함수는 다변량 데이터의 변수 간의 관계를 시각적으로 탐색하고
이해하는 데 유용합니다. 변수들 간의 관계를 한눈에 파악하여
데이터에 대한 직관을 얻을 수 있으며,
이를 통해 추가적인 분석을 수행할 수 있습니다.
Seaborn 라이브러리를 사용하므로 seaborn이 설치되어 있어야 합니다.
'''
```

```
sns.pairplot(selling, height=1.5)
```

Out[17]: <seaborn.axisgrid.PairGrid at 0x167e93790>



```
In [18]: '''
`corr()` 함수는 Pandas 라이브러리에서 제공하는 메서드로,
데이터프레임의 각 변수들 간의 상관계수를 계산하는 기능을 제공합니다.

사용법:
correlation_matrix = dataframe.corr(method='pearson')

인자:
- `method` (옵션): 상관계수를 계산하는 방법을 지정합니다.
  주로 사용되는 방법으로는 'pearson', 'kendall', 'spearman'이 있습니다.
  기본값은 'pearson'입니다.

리턴값:
- 상관계수 행렬(Correlation Matrix)이라고도 불리는 데이터프레임을 반환합니다.
  이 행렬은 변수들 간의 모든 가능한 조합에 대한 상관계수를 포함합니다.

특징:
- `corr()` 함수는 주어진 데이터프레임의 각 변수들 간의 상관계수를 계산합니다.
- 상관계수는 -1에서 1까지의 값을 가지며, 0에 가까울수록 두 변수 간의
  선형적인 관계가 약하거나 없음을 의미하고,
  1이나 -1에 가까울수록 강한 선형적 관계를 나타냅니다.
- `method` 매개변수를 사용하여 상관계수를 계산하는 방법을 지정할 수 있습니다.
  기본적으로 'pearson' 방법이 사용되며, 다른 방법으로 'kendall'과
  'spearman'을 사용할 수 있습니다. 'pearson'은 선형 상관관계를 계산하고,
  'kendall'과 'spearman'은 비선형 상관관계를 계산합니다.
- 반환된 상관계수 행렬은 대칭 행렬이며, 대각선 요소는 항상 1입니다.
  이는 변수 자신과의 상관계수가 항상 1이기 때문입니다.

`corr()` 함수를 사용하면 데이터프레임의 변수들 간의 상관계수를
쉽게 계산할 수 있습니다. 이를 통해 변수들 간의 관계를 빠르게 파악하고
데이터에 대한 이해를 높일 수 있습니다.
'''

selling.corr()
```

Out[18]:

	Price	Bedrooms	HSize	LotSize
Price	1.000000	0.645411	0.747762	0.740874
Bedrooms	0.645411	1.000000	0.846454	0.837430
HSize	0.747762	0.846454	1.000000	0.993615
LotSize	0.740874	0.837430	0.993615	1.000000

```
In [19]: from statsmodels.stats.outliers_influence
import variance_inflation_factor
```

In [20]:

```
'''
코드는 다중공선성(Multicollinearity)을 평가하기 위해
Variance Inflation Factor(VIF)를 계산하는 과정을 수행합니다.
다중공선성은 독립 변수들 간의 높은 상관관계로 인해
회귀 분석 결과의 신뢰도가 저하되는 문제를 나타냅니다.
VIF는 각 독립 변수의 설명력이 다른 독립 변수들로 설명될 수 있는 정도를
나타내는 지표로, 일반적으로 10을 넘으면 다중공선성이 있다고 판단합니다.

아래는 코드의 작동 과정에 대한 설명입니다:
1. `X=selling.drop(['Price'],axis=1)`: 종속 변수를 제외한 모든 독립 변수를
   포함하는 데이터프레임을 생성합니다. 이를 `X`에 할당합니다.
2. `X_design = sm.add_constant(X)`: StatsModels의 `add_constant()` 함수를
   사용하여 독립 변수에 상수항을 추가합니다.
   이는 회귀 모델에서 절편을 고려하기 위해 필요합니다.
   이렇게 수정된 독립 변수를 `X_design`에 할당합니다.
3. `vif = pd.DataFrame()`: 빈 데이터프레임을 생성하여
   VIF 값을 저장할 준비를 합니다.
4. `vif["Features"] = X.columns`: 빈 데이터프레임에 "Features" 열을 추가하고,
   해당 열에 독립 변수들의 이름을 저장합니다.
5. `vif["VIF Factor"] = [...]`: VIF 값을 계산하여 "VIF Factor" 열에 저장합니다.
   이 때, 리스트 컴프리헨션을 사용하여 각 독립 변수에 대한
   VIF 값을 순회하며 계산합니다.
   `variance_inflation_factor()` 함수를 사용하여
   각 독립 변수의 VIF 값을 계산하고, 해당 값을 리스트에 저장합니다.
   `np.round()` 함수를 사용하여 계산된 VIF 값을 소수점 네 자리까지 반올림합니다.
6. 결과적으로 `vif` 데이터프레임에는 각 독립 변수의 이름과 해당 변수의
   VIF 값이 포함되어 있습니다. 이를 통해 다중공선성을 평가하고,
```

VIF 값이 높은 변수들을 식별하여 모델에 적합한 조치를 취할 수 있습니다.

```
'''
```

```
X=selling.drop(['Price'],axis=1)
X_design = sm.add_constant(X)
vif = pd.DataFrame()
vif["Features"] = X.columns
vif["VIF Factor"]
    = [np.round(variance_inflation_factor(X_design.values, i+1),4)
        for i in range(len(X.columns))]
vif
```

Out[20]:

	Features	VIF Factor
0	Bedrooms	3.5400
1	HSize	83.0668
2	LotSize	78.8413

In [21]:

```
X=selling.drop(['Price','LotSize'],axis=1)
X_design = sm.add_constant(X)
vif = pd.DataFrame()
vif["Features"] = X.columns
vif["VIF Factor"]
    = [np.round(variance_inflation_factor(X_design.values, i+1),4)
        for i in range(len(X.columns))]
vif
```

Out[21]:

	Features	VIF Factor
0	Bedrooms	3.5271
1	HSize	3.5271

변수선택

```
In [22]: df = pd.read_csv('copdcat.csv')
df
```

```
Out[22]:
```

	age	sex	FEV1%	Smoke_pack_year	Chol	Comorbid	premium	CATScore
0	53	1	74.13	9.2	170.83	1	59649	7.8
1	48	1	53.24	0.0	168.82	0	47348	9.1
2	55	1	70.39	16.3	193.98	0	49599	7.6
3	62	0	69.87	44.8	215.33	1	58428	14.5
4	48	1	76.41	32.8	184.65	1	50806	11.1
...	...	...	...	...	...	...	...	...
5246	45	1	53.32	0.0	241.90	1	45933	7.1
5247	37	0	57.03	44.4	182.58	0	38004	16.8
5248	56	0	50.61	18.6	198.92	0	66896	7.3
5249	53	1	63.59	45.3	206.71	1	67557	12.2
5250	42	1	64.52	11.9	200.45	0	51339	5.5

5251 rows × 8 columns

```
In [23]: X=df.drop(["CATScore"],axis=1)
y=df["CATScore"]
```

```
In [24]: model_reg = sm.OLS(y,sm.add_constant(X))
result_reg = model_reg.fit()
result_reg.summary()
```

Out[24]:

OLS Regression Results

Dep. Variable:	CATScore	R-squared:	0.740
Model:	OLS	Adj. R-squared:	0.740
Method:	Least Squares	F-statistic:	2130.
Date:	Fri, 01 Mar 2024	Prob (F-statistic):	0.00
Time:	23:19:58	Log-Likelihood:	-10503.
No. Observations:	5251	AIC:	2.102e+04
Df Residuals:	5243	BIC:	2.107e+04
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	9.7637	0.246	39.756	0.000	9.282	10.245
age	0.0147	0.003	4.733	0.000	0.009	0.021
sex	0.2690	0.049	5.446	0.000	0.172	0.366
FEV1%	-0.0787	0.002	-50.431	0.000	-0.082	-0.076
Smoke_pack_year	0.1542	0.001	107.823	0.000	0.151	0.157
Chol	0.0003	0.001	0.387	0.699	-0.001	0.002
Comorbid	0.9392	0.049	19.006	0.000	0.842	1.036
premium	-1.504e-05	1.66e-06	-9.058	0.000	-1.83e-05	-1.18e-05

Omnibus:	1902.692	Durbin-Watson:	1.987
Prob(Omnibus):	0.000	Jarque-Bera (JB):	20452.640
Skew:	1.418	Prob(JB):	0.00

Kurtosis:	12.243	Cond. No.	5.68e+05
------------------	--------	------------------	----------

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 5.68e+05. This might indicate that there are strong multicollinearity or other numerical problems.

In [25]: `import feature_selection as fsel`

후진제거법

In [26]: `'''`

``backwardSelection()`` 함수는 다중 선형 회귀 모델에서 후진 제거(`backward elimination`) 방법을 사용하여 변수 선택을 수행하는 함수입니다. 후진 제거는 모든 독립 변수를 포함한 모델에서 시작하여 가장 덜 유의미한 변수를 하나씩 제거하면서 모델을 재적합하는 방법입니다. 이를 통해 모델의 성능을 유지하면서 변수의 수를 줄일 수 있습니다.

사용법:

```
selected_features = backwardSelection(X, y, significance_level)
```

인자:

- ``X``: 독립 변수들의 데이터를 포함한 특성 행렬입니다.
- ``y``: 종속 변수의 데이터를 포함한 벡터입니다.
- ``significance_level``: 변수를 제거할 때 사용할 유의수준(`significance level`)입니다. 보통 `0.05`나 `0.01`을 사용합니다.

리턴값:

- ``selected_features``: 후진 제거를 통해 선택된 변수들의 인덱스 또는 이름을 담은 리스트 또는 배열입니다.

특징:

- 후진 제거는 모든 독립 변수를 포함한 모델에서 시작하여

- 가장 덜 유의미한 변수를 하나씩 제거하면서 모델을 재적합합니다.
- 각 변수의 유의성은 주어진 유의수준과 비교하여 결정됩니다.
유의수준보다 높은 **p-value**를 가진 변수들이 제거 대상이 됩니다.
- 변수가 제거될 때마다 모델의 성능이 평가되고,
선택된 변수들을 반환합니다.
- 후진 제거를 통해 선택된 변수들을 사용하여
새로운 모델을 구축하고 평가할 수 있습니다.
- 이 방법은 모든 가능한 조합을 고려하지 않기 때문에
최적의 모델을 보장하지는 않습니다.
그러나 변수의 수를 줄이는 데 유용하며,
너무 많은 변수를 고려할 때 과적합을 방지할 수 있습니다.

``backwardSelection()`` 함수를 사용하면 후진 제거 방법을 통해 변수를 선택할 수 있으며, 이를 통해 모델의 복잡성을 줄이고 예측 성능을 향상시킬 수 있습니다.

...

...

``backwardSelection()`` 함수에는 ``model_type``과 ``elimination_criteria``라는 두 가지 추가적인 인자가 있는데, 각각 다음과 같은 역할을 합니다.

``model_type`` 인자:

- ``model_type``은 후진 제거 과정에서 사용할 회귀 모델의 유형을 지정합니다.
- 일반적으로 선형 회귀(Linear Regression), 릿지 회귀(Ridge Regression), 라쏘 회귀(Lasso Regression) 등의 모델을 사용할 수 있습니다.
- 각 모델은 다른 변수 선택 알고리즘을 사용할 수 있으며,
이는 후진 제거 과정에서 각 변수의 유의성을 평가하는 방식을 결정합니다.
- 예를 들어, 릿지 회귀는 L2 정규화를 사용하여 변수의 중요도를 조정하고,
라쏘 회귀는 L1 정규화를 사용하여 희소성을 강조할 수 있습니다.
- 이 인자의 기본값은 일반적으로 선형 회귀를 사용합니다.

``elimination_criteria`` 인자:

- ``elimination_criteria``는 후진 제거 과정에서 변수를 제거하는 기준을 지정합니다.
- 주로 사용되는 기준으로는 **p-value**,
VIF(Variance Inflation Factor) 등이 있습니다.
- ``elimination_criteria``가 **p-value**로 설정된 경우,
해당 변수의 **p-value**가 지정된 유의수준보다 높으면
해당 변수를 제거합니다.

- ``elimination_criteria``가 VIF로 설정된 경우,
해당 변수의 VIF 값이 지정된 임계값을 초과하면 해당 변수를 제거합니다.
- 이 외에도 AIC(Akaike Information Criterion),
BIC(Bayesian Information Criterion) 등의 기준을 사용할 수 있습니다.
- 이 인자의 기본값은 주로 p-value가 사용되며,
일반적으로 0.05의 유의수준이 많이 사용됩니다.

이 두 인자를 설정하여 후진 제거 과정을 조정하고
원하는 결과를 얻을 수 있습니다. 선택한 모델 유형과 제거 기준은
후진 제거 과정에서 변수의 유의성을 평가하는 방식을 결정하므로,
이를 적절히 선택하는 것이 중요합니다.

'''

```
result_back = fsel.backwardSelection(X, y,  
                                     model_type="linear",  
                                     elimination_criteria="aic")  
result_back
```

Character Variables (Dummies Generated, First Dummies Dropped): []

Eliminated : Chol

OLS Regression Results

```
=====
Dep. Variable:          CATScore    R-squared:                0.740
Model:                  OLS         Adj. R-squared:           0.740
Method:                 Least Squares   F-statistic:             2486.
Date:                  Fri, 01 Mar 2024   Prob (F-statistic):       0.00
Time:                  23:19:58         Log-Likelihood:          -10503.
No. Observations:      5251           AIC:                    2.102e+04
Df Residuals:          5244           BIC:                    2.107e+04
Df Model:              6
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
intercept	9.8172	0.203	48.390	0.000	9.419	10.215
age	0.0147	0.003	4.736	0.000	0.009	0.021
sex	0.2689	0.049	5.444	0.000	0.172	0.366
FEV1%	-0.0787	0.002	-50.435	0.000	-0.082	-0.076
Smoke_pack_year	0.1542	0.001	107.833	0.000	0.151	0.157
Comorbid	0.9392	0.049	19.008	0.000	0.842	1.036
premium	-1.504e-05	1.66e-06	-9.055	0.000	-1.83e-05	-1.18e-05

```
=====
Omnibus:                1901.783    Durbin-Watson:           1.987
Prob(Omnibus):          0.000       Jarque-Bera (JB):        20417.644
Skew:                   1.418       Prob(JB):                0.00
Kurtosis:               12.235      Cond. No.                 4.69e+05
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 4.69e+05. This might indicate that there are strong multicollinearity or other numerical problems.

AIC: 21020.090010391912

BIC: 21066.053227087377

Final Variables: ['intercept', 'age', 'sex', 'FEV1%', 'Smoke_pack_year', 'Comorbid', 'premium']

Out[26]: ['intercept', 'age', 'sex', 'FEV1%', 'Smoke_pack_year', 'Comorbid', 'premium']

전진선택법

```
In [27]: '''
`forwardSelection()` 함수는 다중 선형 회귀 모델에서
전진 선택(forward selection) 방법을 사용하여
변수 선택을 수행하는 함수입니다. 전진 선택은 각 변수를 하나씩 추가하여
최적의 변수 집합을 찾는 방법으로,
후진 제거와 반대로 시작점이 없는 방법입니다.

사용법:
selected_features = forwardSelection(X, y, significance_level)

인자:
- `X`: 독립 변수들의 데이터를 포함한 특성 행렬입니다.
- `y`: 종속 변수의 데이터를 포함한 벡터입니다.
- `significance_level`: 변수를 추가할 때
  사용할 유의수준(significance level)입니다.
  보통 0.05나 0.01을 사용합니다.

리턴값:
- `selected_features`: 전진 선택을 통해 선택된 변수들의
  인덱스 또는 이름을 담은 리스트 또는 배열입니다.

특징:
- 전진 선택은 모든 변수를 포함하지 않은 상태에서 시작하여
  가장 유의미한 변수를 하나씩 추가하면서 모델을 구성합니다.
- 각 변수가 추가될 때마다 모델의 성능이 평가되고,
  선택된 변수들을 반환합니다.
- 변수가 추가될 때마다 해당 변수의 유의성은 주어진
  유의수준과 비교하여 결정됩니다.
- 전진 선택은 변수를 추가할 때마다 가능한 모든 조합을 고려하지 않으므로
  최적의 모델을 보장하지는 않습니다.
  그러나 변수의 수를 늘리는 데 유용하며,
  모든 변수를 고려할 때 과적합을 방지할 수 있습니다.
- 선택된 변수들을 사용하여 새로운 모델을 구축하고 평가할 수 있습니다.

전진 선택은 후진 제거와 마찬가지로 변수 선택을 수행하는 방법 중 하나이며,
다중 선형 회귀 모델에서 변수 선택을 자동화하는 데 사용될 수 있습니다.
변수를 추가하는 과정에서 모델의 성능을 평가하고
가장 유의미한 변수들을 선택하여 모델을 구성할 수 있습니다.
'''
```

```
result_for = fsel.forwardSelection(X, y,  
                                   model_type="linear",  
                                   elimination_criteria="aic")  
result_for
```

Character Variables (Dummies Generated, First Dummies Dropped): []

Entered : Smoke_pack_year AIC : 23415.977666326056

Entered : FEV1% AIC : 21489.584865762954

Entered : Comorbid AIC : 21145.80733047592

Entered : premium AIC : 21068.068046043445

Entered : sex AIC : 21040.499068836027

Entered : age AIC : 21020.090010391912

Break : Significance Level

OLS Regression Results

```
=====
Dep. Variable:          CATScore   R-squared:                0.740
Model:                  OLS        Adj. R-squared:             0.740
Method:                 Least Squares   F-statistic:             2486.
Date:                  Fri, 01 Mar 2024   Prob (F-statistic):       0.00
Time:                  23:19:58         Log-Likelihood:          -10503.
No. Observations:      5251           AIC:                    2.102e+04
Df Residuals:          5244           BIC:                    2.107e+04
Df Model:              6
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
intercept	9.8172	0.203	48.390	0.000	9.419	10.215
Smoke_pack_year	0.1542	0.001	107.833	0.000	0.151	0.157
FEV1%	-0.0787	0.002	-50.435	0.000	-0.082	-0.076
Comorbid	0.9392	0.049	19.008	0.000	0.842	1.036
premium	-1.504e-05	1.66e-06	-9.055	0.000	-1.83e-05	-1.18e-05
sex	0.2689	0.049	5.444	0.000	0.172	0.366
age	0.0147	0.003	4.736	0.000	0.009	0.021

```
=====
Omnibus:                1901.783   Durbin-Watson:           1.987
Prob(Omnibus):          0.000     Jarque-Bera (JB):        20417.644
Skew:                   1.418     Prob(JB):                 0.00
Kurtosis:               12.235     Cond. No.                 4.69e+05
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 4.69e+05. This might indicate that there are strong multicollinearity or other numerical problems.

AIC: 21020.090010391912

BIC: 21066.053227087377

Final Variables: ['intercept', 'Smoke_pack_year', 'FEV1%', 'Comorbid', 'premium', 'sex', 'age']

Out[27]: ['intercept', 'Smoke_pack_year', 'FEV1%', 'Comorbid', 'premium', 'sex', 'age']

sklearn 패키지의 RFE 함수 (비추천)

```
In [28]: from sklearn.feature_selection import RFE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
```

```
In [29]: #표준화를 반드시 먼저 수행
scaler = StandardScaler()
X_std = scaler.fit_transform(X)
X_std = pd.DataFrame(X_std)
X_std.columns = X.columns
```

```
In [ ]: '''
`sklearn` 패키지의 `RFE` (Recursive Feature Elimination) 클래스는
재귀적 특성 제거 방법을 사용하여 주어진 모델에 대한
최적의 변수 집합을 선택하는 기능을 제공합니다.
이 방법은 모델의 성능을 평가하고 변수를 제거하여
모델을 다시 구성하는 과정을 반복함으로써 변수의 중요도를 평가합니다.

특징:
- `RFE` 클래스는 재귀적으로 변수를 제거하면서 모델을 다시 적합합니다.
- 변수가 하나씩 제거될 때마다 모델의 성능을 평가하고,
  가장 중요하지 않은 변수를 제거합니다.
- 이 과정은 미리 정의된 변수의 개수나 다른 정지 기준에
  도달할 때까지 반복됩니다.
- 주어진 모델의 최적 변수 집합을 선택하여
  모델의 성능을 최대화합니다.

사용법:
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression

# 모델 초기화
estimator = LinearRegression()
```

```
# RFE 객체 생성
rfe = RFE(estimator, n_features_to_select=5) # 선택할 특성의 개수 설정

# RFE 적합
rfe.fit(X, y)

# 선택된 특성 확인
selected_features = rfe.support_

# 선택된 특성의 순위 확인
feature_ranking = rfe.ranking_

# 선택된 특성 사용
X_selected = X[:, selected_features]
```

인자:

- ``estimator``: 변수 선택에 사용할 모델입니다.
주로 회귀 분석이나 분류 모델을 사용합니다.
- ``n_features_to_select``: 선택할 특성의 개수입니다.
이 값이 지정되지 않은 경우, 절반의 변수가 선택됩니다.
- ``step``: 각 반복에서 제거할 변수의 개수입니다. 기본값은 1입니다.

리턴값:

- ``support_``: 선택된 특성을 나타내는 이진 배열입니다.
- ``ranking_``: 선택된 특성의 순위를 나타내는 배열입니다.
랭킹이 낮을수록 중요도가 높습니다.

``RFE`` 클래스를 사용하면 주어진 모델에 대한 최적의 변수 집합을 선택할 수 있습니다. 이를 통해 모델의 복잡성을 줄이고 예측 성능을 향상시킬 수 있습니다.

...

```
In [30]: model = LinearRegression()
rfe = RFE(model, n_features_to_select=6, step=1)
fit = rfe.fit(X_std, y)

print("Number of Features: ", fit.n_features_)
print("Selected Features: ", X.columns[fit.support_])
```

Number of Features: 6

Selected Features: Index(['age', 'sex', 'FEV1%', 'Smoke_pack_year', 'Comorbid', 'premium'], dtype='object')

In []: